AI Assisted Coding

Assignment – 13.3

M.Rohan || 2303A51545 || Batch:- 8

Task 1: Refactoring – Removing Code Duplication

Objective

To eliminate repeated logic by extracting reusable functions.

Task Description

Use AI assistance to refactor a legacy Python script that contains repeated blocks of code calculating the area and perimeter of rectangles.

Starter (Legacy) Code

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

```python
Lab_13.py > ...
# Refactor the following legacy Python code to remove duplication by creating reusable functions for calculating area and perimeter of a rectangle.
# Improve readability and reusability.

# # Legacy script
# print("Area of Rectangle:", 5 * 10)
# print("Perimeter of Rectangle:", 2 * (5 + 10))
# print("Area of Rectangle:", 7 * 12)
# print("Perimeter of Rectangle:", 2 * (7 + 12))
# print("Area of Rectangle:", 10 * 15)
# print("Perimeter of Rectangle:", 2 * (10 + 15))
# Refactored script with reusable functions
def calculate_area(length, width):
    return length * width

def calculate_perimeter(length, width):
    return 2 * (length + width)
# Example usage
rectangles = [(5, 10), (7, 12), (10, 15)]
for length, width in rectangles:
    area = calculate_area(length, width)
    perimeter = calculate_perimeter(length, width)
    print(f"Area of Rectangle ({length} x {width}): {area}")
    print(f"Perimeter of Rectangle ({length} x {width}): {perimeter}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/N
nt/Lab_13.py"
Area of Rectangle (5 x 10): 50
Perimeter of Rectangle (5 x 10): 30
Area of Rectangle (7 x 12): 84
Perimeter of Rectangle (7 x 12): 38
Area of Rectangle (10 x 15): 150
Perimeter of Rectangle (10 x 15): 50
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>
```

Task 2: Refactoring – Optimizing Loops and Conditionals

Objective

To improve performance by replacing inefficient nested loops with optimized structures.

Task Description

Use AI to analyze and refactor a script that checks the presence of elements using nested loops.

Starter (Legacy) Code

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
found = False
for n in names:
if s == n:
found = True
if found:
print(f"{s} is in the list")
else:
```

print(f"{s} is not in the list")

```python
# Refactor the following Python code to optimize nested loops using set lookups or list comprehensions.
# Improve performance and readability, and show a comparison of time complexity before and after refactoring.

# names = ["Alice", "Bob", "Charlie", "David"]
# search_names = ["Charlie", "Eve", "Bob"]
# for s in search_names:
#     found = False
#     for n in names:
#         if s == n:
#             found = True
#     if found:
#         print(f"{s} is in the list")
#     else:
#         print(f"{s} is not in the list")

# Refactored code using set lookups
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
names_set = set(names)  # Convert list to set for O(1) lookups
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
# Time complexity comparison:
# Before refactoring: O(n * m) where n is the length of names and m is the length of search_names (due to nested loops).
# After refactoring: O(n + m) where n is the time to create the set and m is the time to check each search name in the set (due to O(1) lookups).
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/NI
nt/Lab_13.py"
Charlie is in the list
Eve is not in the list
Bob is in the list
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>
```

Task 3: Refactoring – Extracting Reusable Functions

Objective

To modularize code by extracting calculations into reusable functions.

Task Description

Refactor a legacy script where price and tax calculations are written inline.

Starter (Legacy) Code

price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)

```python
# Refactor the following Python code by extracting reusable logic into a function calculate_total(price).
# Improve structure, remove duplication, and add proper documentation (docstring).
# price = 250
# tax = price * 0.18
# total = price + tax
# print("Total Price:", total)

# price = 500
# tax = price * 0.18
# total = price + tax
# print("Total Price:", total)
# Refactored code with reusable function and documentation
def calculate_total(price):
    """
    Calculate the total price including tax.

    Parameters:
    price (float): The original price of the item.

    Returns:
    float: The total price after adding tax.
    """
    tax = price * 0.18
    total = price + tax
    return total
# Example usage
prices = [250, 500]
for price in prices:
    total_price = calculate_total(price)
    print(f"Total Price for {price}: {total_price}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Py
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Py
Total Price for 250: 295.0
Total Price for 500: 590.0
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>
```

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Objective

To improve maintainability by replacing magic numbers with named constants.

Task Description

Use AI to identify hardcoded values and replace them with constants.

Starter (Legacy) Code

print("Area of Circle:", 3.14159 * (7 ** 2))

print("Circumference of Circle:", 2 * 3.14159 * 7)

```python
# Refactor the following Python code by replacing hardcoded values with named constants (e.g., PI, RADIUS).
# Improve readability and maintainability. and explain the benefits of using named constants in code.

# print("Area of Circle:", 3.14159 * (7 ** 2))
# print("Circumference of Circle:", 2 * 3.14159 * 7)
# Refactored code with named constants
PI = 3.14159
RADIUS = 7
area_of_circle = PI * (RADIUS ** 2)
circumference_of_circle = 2 * PI * RADIUS
print(f"Area of Circle: {area_of_circle}")
print(f"Circumference of Circle: {circumference_of_circle}")
# Benefits of using named constants:
# 1. Readability: Named constants make the code more readable and self-explanatory.
# It is easier to understand what PI and RADIUS represent compared to hardcoded values.
# 2. Maintainability: If the value of a constant needs to be changed (e.g., if we want to use a more precise value of PI),
#  we only need to update it in one place, rather than searching through the code for all occurrences of the hardcoded value.
# 3. Avoiding Magic Numbers: Using named constants helps to avoid "magic numbers" in the code,
# which can be confusing and make the code less clear. It provides context to the values being used.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python313/pyt
nt/Lab_13.py"
Area of Circle: 153.93791
Circumference of Circle: 43.98226
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>
```

Task 5: Refactoring – Improving Variable Naming and Readability

Objective

To enhance readability using descriptive variable names and comments.

Task Description

Refactor a script with unclear variable names.

Starter (Legacy) Code

a = 10

b = 20

c = a * b / 2

print(c)

```python
# Refactor the following Python code by improving variable names and readability.
# Add descriptive names and inline comments while keeping the output the same. and explain the importance of meaningful variable names in code.

# a = 10
# b = 20
# c = a * b / 2
# print(c)
# Refactored code with improved variable names and inline comments
base_length = 10  # Length of the base of the triangle
height = 20       # Height of the triangle
area_of_triangle = (base_length * height) / 2  # Calculate the area of the triangle using the formula (base * height) / 2
print(f"Area of the triangle: {area_of_triangle}")  # Output the area of the triangle
# Importance of meaningful variable names:
# 1. Readability: Meaningful variable names make the code easier to read and understand.
# They provide context about what the variable represents, which helps other developers (or your future self)
#  to quickly grasp the purpose of the variable without needing additional comments.
# 2. Maintainability: When variable names are descriptive, it is easier to maintain and update the code.
# Developers can quickly identify what each variable is used for, which reduces the chances of introducing bugs when making changes.
# 3. Collaboration: In a team environment, meaningful variable names facilitate better communication among developers.
# It allows team members to understand each other's code more easily, leading to more efficient collaboration and code reviews.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Use
nt/Lab_13.py"
Area of the triangle: 100.0
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>
```